فصل ۳: مقدمه‌ای بر SQL

مفاهیم سیستم‌های پایگاه داده - ویرایش هفتم

رئوس مطالب (اسلاید 3.2)

- مرور کلی بر زبان پرس‌وجوی SQL
- تعریف داده در SQL
- ساختار پایه پرس‌وجوهای SQL
- عملیات پایه‌ای تکمیلی
- عملیات مجموعه‌ای (Set Operations)
- مقادیر تهی (Null Values)
- توابع تجمعی (Aggregate Functions)
- پرس‌وجوهای فرعی تودرتو (Nested Subqueries)
- تغییر و اصلاح پایگاه داده

تاریخچه (اسلاید 3.3)

- زبان IBM Sequel به عنوان بخشی از پروژه System R در آزمایشگاه تحقیقاتی IBM San Jose توسعه یافت.
- سپس به زبان پرس‌وجوی ساخت‌یافته (SQL) تغییر نام داد.
- استانداردهای ANSI و ISO برای SQL:
  - SQL-86
  - SQL-89
  - SQL-92
  - SQL:1999 (نام زبان با مشکل سال ۲۰۰۰ یا Y2K سازگار شد!)
  - SQL:2003
- سیستم‌های تجاری بیشتر (اگر نگوییم همه) ویژگی‌های SQL-92 را به علاوه مجموعه‌ای از ویژگی‌های استانداردهای بعدی و ویژگی‌های اختصاصی خودشان ارائه می‌دهند.
  - ممکن است همه مثال‌های اینجا روی سیستم خاص شما کار نکنند.

بخش‌های SQL (اسلاید 3.4)

- **زبان دستکاری داده (DML):** قابلیت پرس‌وجوی اطلاعات از پایگاه داده و همچنین درج (insert)، حذف (delete) و اصلاح (modify) تاپل‌ها (رکوردها) در پایگاه داده را فراهم می‌کند.
- **یکپارچگی (Integrity):** زبان تعریف داده (DDL) شامل دستوراتی برای تعیین قیود یکپارچگی است.
- **تعریف نما (View definition):** DDL شامل دستوراتی برای تعریف نماها (views) است.
- **کنترل تراکنش (Transaction control):** شامل دستوراتی برای تعیین نقطه شروع و پایان تراکنش‌هاست.
- **اس‌کیوال تعبیه‌شده و پویا (Embedded and dynamic SQL):** نحوه قرارگیری (تعبیه) دستورات SQL در زبان‌های برنامه‌نویسی همه‌منظوره را تعریف می‌کند.
- **مجوز دسترسی (Authorization):** شامل دستوراتی برای تعیین حقوق دسترسی به رابطه‌ها (جداول) و نماها است.

زبان تعریف داده (DDL) (اسلاید 3.5)

زبان تعریف داده SQL (یا DDL) اجازه مشخص کردن اطلاعات مربوط به رابطه‌ها را می‌دهد، از جمله:

- طرح‌واره (schema) برای هر رابطه.
- نوع مقادیر مرتبط با هر صفت (ویژگی/ستون).
- قیود یکپارچگی.
- مجموعه شاخص‌هایی (indices) که باید برای هر رابطه نگهداری شوند.
- اطلاعات امنیتی و مجوزها برای هر رابطه.
- ساختار ذخیره‌سازی فیزیکی هر رابطه روی دیسک.

انواع دامنه در SQL (اسلاید 3.6)

- `char(n)`: رشته کاراکتری با طول ثابت برابر با nnn که توسط کاربر مشخص می‌شود.
- `varchar(n)`: رشته‌های کاراکتری با طول متغیر، با حداکثر طول nnn که توسط کاربر مشخص می‌شود.
- `int`: عدد صحیح (زیرمجموعه‌ای متناهی از اعداد صحیح که وابسته به ماشین است).
- `smallint`: عدد صحیح کوچک (زیرمجموعه‌ای وابسته به ماشین از نوع دامنه اعداد صحیح).
- `numeric(p,d)`: عدد با ممیز ثابت، با دقت ppp رقم (مشخص شده توسط کاربر) که ddd رقم آن در سمت راست ممیز اعشار قرار دارد. (مثلاً `numeric(3,1)` اجازه می‌دهد مقدار 44.5 دقیقاً ذخیره شود، اما 444.5 یا 0.32 را نمی‌پذیرد).
- `real, double precision`: اعداد اعشاری با ممیز شناور و ممیز شناور با دقت مضاعف، با دقتی که وابسته به ماشین است.
- `float(n)`: عدد اعشاری با ممیز شناور، با دقت حداقل nnn رقم که توسط کاربر مشخص می‌شود.

- موارد بیشتر در فصل ۴ بررسی می‌شوند.

ساختار Create Table (اسلاید 3.7)

یک رابطه در SQL با استفاده از دستور `create table` تعریف می‌شود:

```
sql
create table r
 (A1 D1,
  A2 D2,
  ...,
  An Dn,
  (integrity-constraint1),
  ...,
  (integrity-constraintk))
```

- حرف `r` نام رابطه (جدول) است.
- هر AiA_iAi نام یک صفت در طرح‌واره رابطه rrr است.
- متغیر DiD_iDi نوع داده مقادیر در دامنه صفت AiA_iAi است.
- **مثال:**

```
sql
create table instructor (
 ID char(5),
 name varchar(20),
 dept_name varchar(20),
 salary numeric(8,2))
```

قیود یکپارچگی در Create Table (اسلاید 3.8)

- انواع قیود یکپارچگی:
  - کلید اصلی (primary key)
  - کلید خارجی (foreign key) … ارجاع می‌دهد به جدول rrr
  - خالی نبودن (not null)

- SQL از هرگونه به‌روزرسانی پایگاه داده که ناقض یک قید یکپارچگی باشد، جلوگیری می‌کند.
- **مثال:**

```
sql
 create table instructor (
 ID char(5),
 name varchar(20) not null,
 dept_name varchar(20),
 salary numeric(8,2),
 primary key (ID),
 foreign key (dept_name) references department);
```

چند تعریف رابطه دیگر (اسلاید 3.9 و 3.10)

(در این بخش کدهای ساخت جداول student (دانشجو)، takes (دروس اخذ شده) و course (درس) با تعیین کلیدهای اصلی و خارجی آورده شده است).

به‌روزرسانی جداول (اسلاید 3.11)

- **درج (Insert):** `insert into instructor values ('10211', 'Smith', 'Biology', 66000);`
- **حذف (Delete):**
  - حذف تمام تاپل‌ها از جدول دانشجو: `delete from student`
- **حذف جدول (Drop Table):** `drop table r`
- **تغییر جدول (Alter):**
  - `alter table r add A D` (که در آن A نام صفت جدیدی است که به رابطه r اضافه می‌شود و D دامنه/نوع داده A است. به تمام تاپل‌های موجود در رابطه، مقدار null برای این صفت جدید اختصاص داده می‌شود.)
  - `alter table r drop A` (که در آن A نام صفتی در رابطه r است. حذف صفات توسط بسیاری از پایگاه‌های داده پشتیبانی نمی‌شود.)

ساختار پایه پرس‌وجو (اسلاید 3.12)

- یک پرس‌وجوی متداول SQL این فرم را دارد:

```
sql
select A1, A2, ..., An
from r1, r2, ..., rm
where P
```

- AiA_iAi نشان‌دهنده یک صفت است.
- RiR_iRi نشان‌دهنده یک رابطه (جدول) است.
- PPP یک گزاره (شرط) است.
- خروجی و نتیجه یک پرس‌وجوی SQL، خودش یک رابطه (جدول) است.

لطفاً اطلاع دهید تا ترجمه بخش بعدی (شروع از ساختار Select و شرط Where تا پایان توابع تجمعی) را برایتان ارسال کنم.

thumb_upthumb_down

ارسال کن

lightbulb ▼مشاهده مراحل استنتاج

در ادامه، ترجمه بخش دوم (اسلایدهای ۳.۱۳ تا ۳.۳۴) که شامل ساختار پرس‌وجوها، عملیات رشته‌ای، مجموعه‌ای و توابع تجمعی است، تقدیم شما می‌شود:

بند Select (اسلاید 3.13)

- بند `select` صفات (ستون‌های) مورد نظر برای نمایش در نتیجه پرس‌وجو را لیست می‌کند.
  - این بخش معادل عملیات پرتو (Projection) در جبر رابطه‌ای است.
- **مثال:** یافتن نام تمام اساتید:

```
sql
select name
from instructor
```

- **نکته:** نام‌ها در SQL به بزرگی و کوچکی حروف حساس نیستند (Case Insensitive). یعنی می‌توانید از حروف بزرگ یا کوچک استفاده کنید (مثلاً `Name ≡ NAME ≡ name`). برخی افراد در کدهایی که ما با فونت بولد می‌نویسیم، از حروف بزرگ استفاده می‌کنند.

بند Select (ادامه) (اسلاید 3.14)

- زبان SQL اجازه وجود رکوردهای تکراری در جداول و همچنین در نتایج پرس‌وجو را می‌دهد.
- برای اجبار به حذف مقادیر تکراری، کلمه کلیدی `distinct` را بعد از `select` قرار دهید.
- **مثال:** یافتن نام دپارتمان تمام اساتید و حذف تکراری‌ها:

```
sql
select distinct dept_name
from instructor
```

- کلمه کلیدی `all` مشخص می‌کند که تکراری‌ها نباید حذف شوند (حالت پیش‌فرض).

بند Select (ادامه) (اسلاید 3.15)

- علامت ستاره `*` در بند select به معنای "تمام صفات" است.

```
sql
select *
from instructor
```

- یک صفت می‌تواند یک مقدار ثابت (Literal) بدون بند from باشد: `select '437'`
  - نتیجه یک جدول با یک ستون و یک سطر با مقدار "437" است. با کلمه `as` می‌توان به ستون نام داد:
    `select '437' as FOO`
- یک صفت می‌تواند یک مقدار ثابت با بند from باشد: `select 'A' from instructor`
  - نتیجه یک جدول با یک ستون و NNN سطر (تعداد تاپل‌های جدول اساتید) است که هر سطر مقدار "A" دارد.

بند Select (ادامه) (اسلاید 3.16)

- بند select می‌تواند شامل عبارات محاسباتی شامل عملگرهای +++، −-−، ***، و /// باشد که روی ثابت‌ها یا صفات تاپل‌ها عمل می‌کنند.
- **پرس‌وجوی زیر:**

```
sql
 select ID, name, salary/12
 from instructor
```

رابطه‌ای را برمی‌گرداند که مشابه جدول instructor است، با این تفاوت که مقدار صفت salary (حقوق) بر 12 تقسیم شده است.

- می‌توان با بند `as` نام ستون جدید را مشخص کرد: `salary/12 as monthly_salary`

بند Where (اسلاید 3.17)

- بند `where` شرایطی را که نتیجه باید برآورده کند مشخص می‌کند.
  - معادل گزاره انتخاب (Selection) در جبر رابطه‌ای است.
- برای یافتن تمام اساتید دپارتمان علوم کامپیوتر:

```
sql
select name
from instructor
where dept_name = 'Comp. Sci.'
```

- SQL استفاده از پیوندهای منطقی `and`، `or` و `not` را مجاز می‌داند.
- عملوندهای این پیوندهای منطقی می‌توانند عباراتی شامل عملگرهای مقایسه‌ای <<<، =>=>=>، >>>، <=<=<=، === و <><><> باشند.
- مقایسه‌ها را می‌توان روی نتایج عبارات محاسباتی نیز اعمال کرد. (مثال: حقوق بیشتر از ۷۰۰۰۰).

بند From (اسلاید 3.18)

- بند `from` رابطه‌های درگیر در پرس‌وجو را لیست می‌کند.
  - معادل عملیات حاصلضرب دکارتی (Cartesian product) در جبر رابطه‌ای.
- یافتن حاصلضرب دکارتی instructor در teaches:

```
sql
select *
from instructor, teaches
```

- این کار تمام جفت‌های ممکن از اساتید و دروس تدریس‌شده را تولید می‌کند. برای صفات مشترک (مثل ID)، نام ستون با نام جدول ترکیب می‌شود (مثل `instructor.ID`).
- حاصلضرب دکارتی به تنهایی خیلی مفید نیست، اما در ترکیب با شرط where بسیار کاربردی است.

عملگر تغییر نام (Rename) (اسلاید 3.20)

- زبان SQL اجازه تغییر نام رابطه‌ها و صفات را با استفاده از کلمه `as` می‌دهد: `old-name as new-name`
- **مثال:** یافتن نام اساتیدی که حقوقشان از حداقل یک استاد در علوم کامپیوتر بیشتر است.

```
sql
select distinct T.name
from instructor as T, instructor as S
where T.salary > S.salary and S.dept_name = 'Comp. Sci.'
```

- کلمه کلیدی `as` اختیاری است و می‌تواند حذف شود: `instructor T` معادل `instructor as T` است.

عملیات روی رشته‌ها (اسلایدهای 3.22 و 3.23)

- SQL شامل عملگر تطبیق رشته با استفاده از `like` است که از دو کاراکتر خاص استفاده می‌کند:
  - درصد (`%`): با هر زیررشته‌ای مطابقت دارد.
  - خط زیرین (`_`): با دقیقاً یک کاراکتر مطابقت دارد.
- **مثال:** اساتیدی که نامشان شامل "dar" است: `%where name like '%dar'`
- برای جستجوی خود کاراکتر % از کلمه کلیدی escape استفاده می‌شود.
- الگوها به بزرگی و کوچکی حروف حساس هستند (در اکثر دیتابیس‌ها). SQL عملگرهای دیگری مانند الحاق (`||`)، تبدیل حروف و طول رشته را نیز پشتیبانی می‌کند.

مرتب‌سازی نتایج (اسلاید 3.24)

- لیست کردن نام اساتید به ترتیب الفبا:

```
sql
select distinct name
from instructor
order by name
```

- می‌توان از `desc` برای ترتیب نزولی و `asc` برای صعودی (پیش‌فرض) استفاده کرد.
- مرتب‌سازی روی چند ویژگی نیز امکان‌پذیر است: `order by dept_name, name`

گزاره‌های بند Where (اسلاید 3.25)

- شامل عملگر `between` برای بازه‌ها (مثال: `salary between 90000 and 100000`).
- مقایسه تاپل‌ها (چندتایی‌ها):

```
where (instructor.ID, dept_name) = (teaches.ID, 'Biology');
```

عملیات مجموعه‌ای (اسلایدهای 3.26 و 3.27)

- **اجتماع (Union):** دروس ارائه شده در پاییز ۲۰۱۷ یا بهار ۲۰۱۸.
- **اشتراک (Intersect):** دروس ارائه شده در پاییز ۲۰۱۷ و بهار ۲۰۱۸.
- **تفاضل (Except):** دروس ارائه شده در پاییز ۲۰۱۷ اما نه در بهار ۲۰۱۸.
- این عملیات به طور خودکار تکراری‌ها را حذف می‌کنند. برای حفظ تکراری‌ها از `union all`، `intersect all` و `except all` استفاده می‌شود.

مقادیر تهی (Null Values) (اسلایدهای 3.28 و 3.29)

- مقدار `null` نشان‌دهنده یک مقدار ناشناخته یا عدم وجود مقدار است.
- نتیجه هر عبارت محاسباتی با null برابر با null است (مثال: null+5null + 5null+5 می‌شود null).
- برای بررسی آن از `is null` و `is not null` استفاده می‌شود.
- در مقایسه‌ها (مثل null>5null > 5null>5) نتیجه "ناشناخته" (unknown) است و SQL از منطق سه‌ارزشی برای and, or, not در برخورد با unknown استفاده می‌کند.

توابع تجمعی (Aggregate Functions) (اسلایدهای 3.30 تا 3.34)

- این توابع روی مجموعه‌ای از مقادیر یک ستون عمل کرده و یک مقدار برمی‌گردانند:

- avg: میانگین
- min: حداقل
- max: حداکثر
- sum: مجموع
- count: تعداد

- **گروه‌بندی (Group By):** برای اعمال توابع تجمعی روی گروه‌های مختلف. مثال: یافتن میانگین حقوق اساتید در هر دپارتمان.

```
sql
select dept_name, avg (salary) as avg_salary
from instructor
group by dept_name;
```

- **بند Having:** برای اعمال شرط روی گروه‌ها (پس از گروه‌بندی اعمال می‌شود، برخلاف where که قبل از تشکیل گروه‌ها اعمال می‌شود).
  - مثال: دپارتمان‌هایی که میانگین حقوقشان بیشتر از ۴۲۰۰۰ است: `having avg (salary) > 42000`

در صورت تمایل بفرمایید تا بخش سوم (از اسلاید 3.35 مربوط به Subqueries تا انتهای فصل) را برایتان ترجمه کنم.

thumb_upthumb_down

ادامه بده

lightbulb ▼مشاهده مراحل استنتاج

در ادامه، بخش سوم و پایانی این فصل که شامل پرس‌وجوهای فرعی (Subqueries) و دستورات تغییر پایگاه داده (Database Modification) است، تقدیم می‌شود:

پرس‌وجوهای فرعی تودرتو (Nested Subqueries)

زبان SQL مکانیزمی برای قرار دادن یک پرس‌وجو (Select-From-Where) در داخل یک پرس‌وجوی دیگر فراهم می‌کند که به آن پرس‌وجوی فرعی (Subquery) می‌گویند. این پرس‌وجوها معمولاً در بند `where` استفاده می‌شوند:

- **عضویت در مجموعه (Set Membership):**
- از عملگرهای `in` و `not in` برای بررسی اینکه آیا یک مقدار در مجموعه‌ای از مقادیر (تولید شده توسط یک پرس‌وجوی فرعی) وجود دارد یا خیر، استفاده می‌شود.
- *مثال:* یافتن دروسی که در پاییز ۲۰۱۷ ارائه شده‌اند و در بهار ۲۰۱۸ نیز ارائه شده‌اند:

```
where course_id in (select course_id from section where ...)
```

- **مقایسه مجموعه‌ای (Set Comparison):**
- **عملگر `some` (یا `any`):** برای مقایسه با "حداقل یکی" از اعضای مجموعه. مثلاً `> some` یعنی "بزرگتر از حداقل یکی از اعضا".
- **عملگر `all`:** برای مقایسه با "همه" اعضای مجموعه. مثلاً `> all` یعنی "بزرگتر از همه اعضا" (بزرگتر از ماکزیمم).
- **تست خالی بودن مجموعه (Empty Relations):**
- از عملگر `exists` برای بررسی اینکه آیا پرس‌وجوی فرعی حداقل یک تاپل (رکورد) برمی‌گرداند یا خیر، استفاده می‌شود. `not exists` برای بررسی خالی بودن مجموعه است.
- بسیار کاربردی برای پرس‌وجوهای همبسته (Correlated Subqueries) که در آن‌ها پرس‌وجوی داخلی از متغیرهای پرس‌وجوی بیرونی استفاده می‌کند.
- **تست تکراری نبودن (Test for Absence of Duplicate Tuples):**
- ساختار `unique` (و `not unique`) بررسی می‌کند که آیا در نتیجه پرس‌وجوی فرعی، رکوردهای تکراری وجود دارد یا خیر (این ویژگی در همه پایگاه‌های داده پشتیبانی نمی‌شود).

پرس‌وجوهای فرعی در سایر بخش‌ها

- **پرس‌وجوی فرعی در بند `from`:** می‌توان کل یک پرس‌وجوی `select` را در بند `from` نوشت و با استفاده از `as` به آن یک نام موقت اختصاص داد. در واقع نتیجه آن پرس‌وجو مانند یک جدول مجازی عمل می‌کند.
- **بند `with`:** روشی بسیار خواناتر برای تعریف جدول‌های موقت است که فقط برای همان پرس‌وجو معتبرند. این بند قبل از `select` اصلی نوشته می‌شود.
- **پرس‌وجوهای فرعی اسکالر (Scalar Subqueries):** پرس‌وجویی که به طور قطع فقط یک سطر و یک ستون (یک مقدار واحد) برمی‌گرداند، می‌تواند در هر جایی که انتظار یک متغیر می‌رود (مثل بند `select` یا `where`) استفاده شود.

تغییر و اصلاح پایگاه داده (Database Modification)

علاوه بر استخراج اطلاعات، SQL برای تغییر داده‌ها نیز استفاده می‌شود:

- **حذف داده (Deletion):**
- فقط می‌توان کل یک تاپل (سطر) را حذف کرد، نه مقادیر یک صفت خاص.
- ساختار: `delete from r where P`
- اگر شرط `where` نوشته نشود، تمام رکوردهای جدول حذف می‌شوند (`delete from instructor`).
- **درج داده (Insertion):**
- وارد کردن یک رکورد منفرد: `insert into instructor values ('10211', 'Smith', 'Biology', 66000)`
- می‌توان ترتیب ستون‌ها را مشخص کرد: `insert into instructor (ID, name, dept_name, salary) values (...)`
- **درج بر اساس پرس‌وجو:** می‌توان نتیجه یک `select` را مستقیماً وارد یک جدول کرد. مثلاً درج تمام اساتید به عنوان دانشجو با تعداد واحد صفر:

```
insert into student select ID, name, dept_name, 0 from instructor
```

- **به‌روزرسانی داده (Updates):**
- برای تغییر مقدار یک یا چند صفت در رکوردهای موجود بدون تغییر کل رکورد.
- ساختار: `update r set A = ... where P`
- *مثال:* افزایش ۵ درصدی حقوق تمام اساتید: `update instructor set salary = salary * 1.05`
- **دستور `case` در به‌روزرسانی:** برای اعمال به‌روزرسانی‌های شرطی مختلف در یک دستور استفاده می‌شود. مثلاً افزایش ۵ درصدی برای حقوق‌های زیر ۱۰۰ هزار و افزایش ۳ درصدی برای بقیه:

```
sql
      update instructor
      set salary = case
                      when salary <= 100000 then salary * 1.05
                      else salary * 1.03
                   end
```

این بخش پایان‌بخش مفاهیم فصل ۳ (مقدمه‌ای بر SQL) بود. در صورت داشتن هرگونه سوال یا نیاز به توضیح بیشتر درباره هر یک از این دستورات، در خدمت شما هستم.